

Regularization for Deep Networks

Scaler School of Technology | Deep Learning I | Week 3 — Session 1/2 | Post-Lecture Review & Practice



Key Takeaways

1 Overfitting is the Default

Deep networks have far more parameters than training samples. Without regularization, they will memorize training data (including noise) rather than learning generalizable patterns.

2 L2 / Weight Decay

Adding $\lambda \sum w_i^2$ to the loss penalizes large weights, forcing simpler models. In Adam, use **AdamW** (decoupled weight decay) — standard L2 and weight decay are not equivalent in adaptive optimizers.

3 Dropout & The Regularization Toolkit

Dropout randomly zeros neurons during training, creating an implicit ensemble. Combined with early stopping (stop when val loss rises) and data augmentation (free synthetic data), these form your complete anti-overfitting toolkit.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why can a network with 11M parameters perfectly fit 50K randomly labeled images?
- Q2.** Write the SGD weight update rule with weight decay. What does the “decay” part do geometrically?
- Q3.** At test time, should dropout be active? What happens if you forget to call `model.eval()`?
- Q4.** Explain why dropout acts as an ensemble method. How many “sub-networks” does a single dropout-trained model approximate?
- Q5.** Your model has 99% train accuracy but 72% val accuracy. List the regularization techniques you’d try, in order.

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Weight Decay Sensitivity

Train a 3-layer MLP (hidden sizes: 256, 128) on the **make_moons** dataset (500 samples, noise=0.3). Train 4 versions with `weight_decay` $\in \{0, 10^{-4}, 10^{-2}, 0.1\}$. For each: (a) plot train/val loss curves, (b) plot the decision boundary. Identify the sweet spot.

Exercise 2: Dropout Rate Comparison

Using the same architecture, add `nn.Dropout` after each hidden layer. Train with dropout rates $\in \{0, 0.2, 0.5, 0.7\}$. Compare final validation accuracy. Also verify: run the same input through the model 10 times in `train()` mode vs `eval()` mode — what do you observe?

Exercise 3: Data Augmentation on CIFAR-10

Load CIFAR-10 (first 5000 images only). Train a small CNN **without** augmentation for 30 epochs. Then train with `RandomHorizontalFlip` + `RandomCrop(32, padding=4)` + `ColorJitter`. Compare val accuracy. Plot 8 augmented versions of a single image.

Challenge Problem

The Regularization Ablation Study

Train a ResNet-18 on CIFAR-10 (full dataset) with the following configurations and report val accuracy for each:

1. Baseline: no regularization at all (`weight_decay=0`, no dropout, no augmentation)
2. Add only weight decay (`1e-4`)
3. Add only data augmentation (flips + crops)
4. Add only dropout (0.3 after FC layers)
5. All combined: weight decay + augmentation + dropout + early stopping

Expected output: A table showing val accuracy for each config. Which single technique helps most? Does combining all of them beat any individual one?

Bonus: Plot training curves for all 5 configs on the same plot.

- **Memorization in DNNs** — Zhang et al., 2017, “*Understanding Deep Learning Requires Rethinking Generalization.*” Deep nets can fit random labels — foundational for understanding why regularization matters.
- **Dropout** — Srivastava et al., 2014, “*Dropout: A Simple Way to Prevent Neural Networks from Overfitting.*” The original paper. Read Sections 1–4 for theory, Section 7 for practical tips.
- **AdamW** — Loshchilov & Hutter, 2019, “*Decoupled Weight Decay Regularization.*” Why $L2 \neq$ weight decay in Adam, and the fix.
- **Data Augmentation** — Shorten & Khoshgoftaar, 2019, “*A Survey on Image Data Augmentation for Deep Learning.*” Reference for choosing augmentations by domain.

Online Resources & Tools

- **PyTorch nn.Dropout** — Official docs with examples of train/eval mode behavior
- **PyTorch Lightning EarlyStopping** — Callback that handles early stopping automatically
- **torchvision.transforms** — All built-in image augmentations with visual examples
- **Weights & Biases** — Track train/val curves across experiments to spot overfitting
- **Albumentations** — Fast augmentation library with 70+ transforms for images
- **CS231n Notes on Regularization** — Stanford lecture notes with clear diagrams

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about bias-variance tradeoff?** Rewatch StatQuest’s “Bias and Variance” video (12 min). Overfitting = high variance, regularization reduces variance at the cost of slight bias increase.
- **Fuzzy on Ridge Regression ($L2$)?** Review your ML course notes on Ridge. The penalty term $\lambda \sum w^2$ is identical — we just apply it to neural net weights now.
- **Not sure what `model.train()` vs `model.eval()` does?** Read the PyTorch docs on `Module.train()` — it affects Dropout and BatchNorm layers. Always switch modes correctly.
- **Confused about ensembles?** Recall Random Forests from ML: many weak models averaged together. Dropout creates a similar effect inside one network — each forward pass is a different “weak model”.

Answers — Test Your Understanding

- A1.** 220 params per sample — enough degrees of freedom to memorize any arbitrary mapping, including random labels.
- A2.** $w \leftarrow (1 - \eta\lambda) \cdot w - \eta \cdot \nabla L$. The $(1-\eta\lambda)$ factor shrinks weights toward zero each step, pulling the weight vector closer to the origin.
- A3.** No. If you forget `model.eval()`, dropout is still active at inference — each forward pass gives a different (noisy) output.
- A4.** Each dropout mask = a different sub-network. With n neurons, $p=0.5$: 2^n sub-networks trained simultaneously. Inference with all neurons $\approx$ averaging all sub-networks.
- A5.** (1) Data augmentation, (2) weight decay (AdamW, $\lambda=10^{-4}$), (3) early stopping, (4) dropout ($p=0.3-0.5$). Still overfitting? More data or smaller model.